```
#Import the required Python libraries for Linear Regression.
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np
```

```
# Create/load a dataset containing one independent and one dependent variable.
df = pd.read_csv('placement.csv')
```

```
#Display the first five records of the dataset.
df.head()
```

	cgpa	package
0	6.89	3.26
1	5.12	1.98
2	7.82	3.25
3	7.42	3.67
4	6.94	3.57

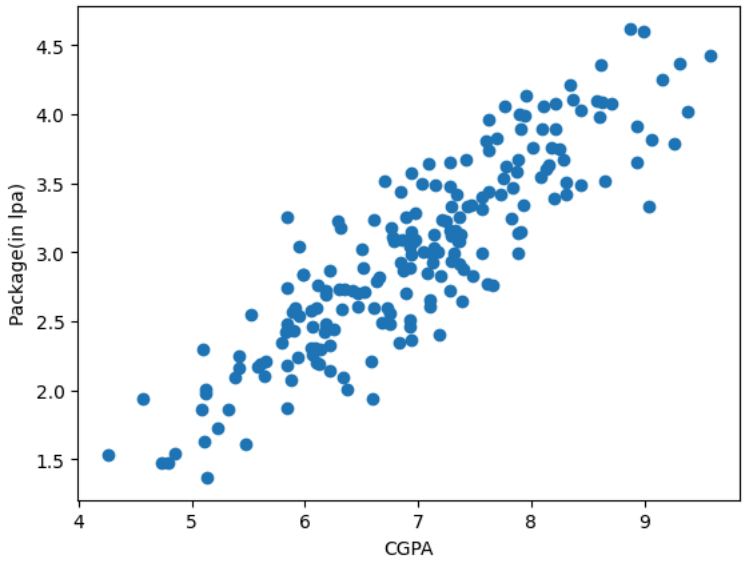
Next steps: Generate code with df | New interactive sheet

```
df.shape
```

```
(200, 2)
```

```
# Visualize the relationship between X and Y using a scatter plot.
plt.scatter(df['cgpa'],df['package'])
plt.xlabel('CGPA')
plt.ylabel('Package(in lpa)')
```

```
Text(0, 0.5, 'Package(in lpa)')
```



Separate input column and output columns

X- input column

y-output column

```
X = df.iloc[:,0:1]
y = df.iloc[:,-1]
```

y

	package
0	3.26
1	1.98
2	3.25
3	3.67
4	3.57
...	...
195	2.46
196	2.57
197	3.24
198	3.96
199	2.33

200 rows × 1 columns

dtype: float64

X

	cgpa
0	6.89
1	5.12
2	7.82
3	7.42
4	6.94
...	...
195	6.93
196	5.89
197	7.21
198	7.63
199	6.22

200 rows × 1 columns

Next steps: Generate code with X | New interactive sheet

Divide data into train test

```
#Split the dataset into training and testing sets.
from sklearn.model_selection import train_test_split
X_train,X_test,y_train,y_test = train_test_split(X,y,test_size=0.2,random_state=2)
```

Import linear **regression**

```
#Train a Linear Regression model.
from sklearn.linear_model import LinearRegression
```

create object of linear gression class

```
lr = LinearRegression()
```

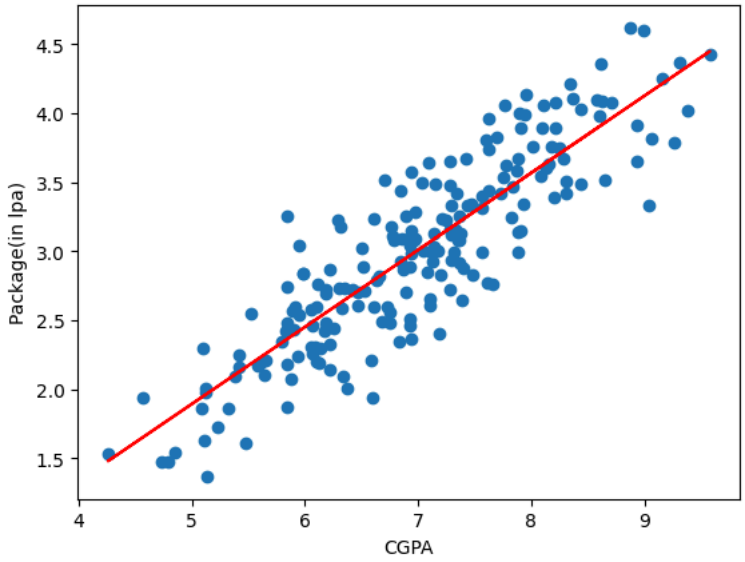
Train the model

```
lr.fit(X_train,y_train)
```

```
▾ LinearRegression
LinearRegression()
```

```
plt.scatter(df['cgpa'],df['package'])
plt.plot(X_train,lr.predict(X_train),color='red')
plt.xlabel('CGPA')
plt.ylabel('Package(in lpa)')
```

```
Text(0, 0.5, 'Package(in lpa)')
```



Display the coefficient and intercept.

```
print(lr.intercept_)
print(lr.coef_)
```

```
-0.8961119222429144
[0.55795197]
```

```
from sklearn.metrics import mean_absolute_error,mean_squared_error,r2_score
```

```
y_pred = lr.predict(X_test)
```

```
y_pred
```

```
array([3.89111601, 3.09324469, 2.38464568, 2.57434935, 1.6537286 ,
       1.77647803, 2.07219258, 2.93143862, 3.76278706, 2.93701814,
       4.09197872, 3.51170867, 2.97049525, 2.40138424, 3.18809652,
       3.46707251, 1.94386362, 3.24389172, 2.97607477, 3.41685683,
       2.55761079, 3.16577844, 2.85890486, 3.12114229, 3.68467378,
       2.8700639 , 3.49497011, 3.34432308, 3.91901361, 1.96060218,
       3.65119666, 3.2104146 , 3.74046898, 2.7863711 , 2.78079158,
       3.27178932, 3.52844723, 2.61340599, 2.65804215, 2.71383735])
```

```
y_test.values
```

```
array([4.1 , 3.49, 2.08, 2.33, 1.94, 1.48, 1.86, 3.09, 4.21, 2.87, 3.65,
       4.  , 2.89, 2.6 , 2.99, 3.25, 1.86, 3.67, 2.37, 3.42, 2.48, 3.65,
       2.6 , 2.83, 4.08, 2.56, 3.58, 3.81, 4.09, 2.01, 3.63, 2.92, 3.51,
       1.94, 2.21, 3.34, 3.34, 3.23, 2.01, 2.61])
```

```
print("MAE",mean_absolute_error(y_test,y_pred))
```

```
MAE 0.2884710931878175
```

```
print("MSE",mean_squared_error(y_test,y_pred))
```

```
MSE 0.12129235313495527
```

```
print("RMSE",np.sqrt(mean_squared_error(y_test,y_pred)))
```

```
RMSE 0.34827051717731616
```

```
print("MSE",r2_score(y_test,y_pred))
r2 = r2_score(y_test,y_pred)
```

```
MSE 0.780730147510384
```

Adjusted R-squared is a modified statistical measure that evaluates how well a regression model fits the data while penalizing the addition of unnecessary independent variables

$$R^2 = 1 - \left(1 - R^2\right) \cdot \frac{n-1}{n-k-1}$$

Where:

n = number of observations

k = number of independent variables (predictors)

R2 = the original R-squared

```
# Adjusted R2 score
X_test.shape
```

```
(40, 1)
```

```
1 - ((1-r2)*(40-1)/(40-1-1))
```

```
0.7749598882343415
```

```
Start coding or generate with AI.
```